CS383 · SOFTWARE ENGINEERING

Lecture 4 Agile & Scrum

Midterm summary

Muharib

What's inside

01 Why did Agile appear?

02 Agile methodology

03 Extreme Programming (XP)

04 User stories

05 Plan-driven vs Agile

06 Scrum

07 Sprint calculations

08 Summary of the four models

09 How does it appear in the midterm?

10 Common mistakes

11 Test yourself

01 Why did Agile appear?

In practice, companies mix the general models to fit what the project needs. The choice of model is affected by: contracts and regulations, the team's experience with the model, and the models used in related projects.

Today software is built quickly and requirements change quickly. So specification, design and implementation have become interleaved, and the system is built in cycles (**Iterations**).

The problem

In this situation, the **Plan-driven** approach carries extra load (**Overhead**) from procedures and documents.

02 Agile methodology

- The focus is on the code, not on design and documents.
- Small releases to the customer every 2 to 3 weeks.
- Suitable for small and medium-sized products.
- Needs a committed customer who takes part in the development.

The five principles

Principle	Meaning
Customer involvement	The customer takes part throughout development. They give the requirements, prioritize them and evaluate each iteration.
Incremental delivery	The software is built in increments, and the customer specifies the requirements of each increment.
People not process	We rely on the skills of the team and let them work in their own way.
Embrace change	We expect the requirements to change, and we design the system so it can cope with that.
Maintain simplicity	We focus on simplicity in the software and in the way of working.

Weaknesses of Agile

- It is hard to keep the customer interested all the time.
- Some team members are not suited to intensive work.
- A new member needs documents to understand the system.
- Changing priorities is hard when there are many stakeholders.
- Simplicity itself needs extra work.

03 Extreme Programming (XP)

An Agile method that takes iterative development to the extreme. Its cycle:

```
Select user stories  →  Break down stories to tasks  →  Plan release
```

```
Develop / integrate / test  →  Release software  →  Evaluate system
```

The eight practices

#	Practice	Meaning
1	**Incremental planning**	Requirements are written on story cards and ordered by priority.
2	**Small releases**	Small, frequent releases every 2 to 3 weeks.
3	**Simple design**	A design that covers the current need only.
4	**Test-first development**	An automated test for every function that is added.
5	**Refactoring**	Improving the code continuously.
6	**Pair programming**	Two programmers review and tidy each other's code.
7	**Continuous integration**	Every finished task is integrated into the system, provided that all tests pass.
8	**On-site customer**	The customer is available all the time.

Appeared in the midterm

Two MCQ questions: the practice that improves the code continuously is **Refactoring**. And one of the core principles of XP is **Continuous feedback**.

Two notes from the slides

- Incremental development damages the structure of the code over time. XP solves this with continuous **Refactoring**.
- Many automated tests do not mean that testing is complete. The programmer may take shortcuts and write incomplete tests.

04 User stories

The project owner and the developers write the **User stories**. Each story has three parts:

Part	Meaning
Who	The user role: customer, employee, system administrator.
What	The goal: what function is required.
Why	The reason: why they need it.

> As a user, I want a cascade action warning before updating or deleting a record, because I want to avoid dependency problems.

Each story can also be given an estimated number of points (**Story points**).

05 Plan-driven vs Agile

In **Plan-driven**, the requirements are completed first and written in a document, and then come design and implementation. In **Agile**, requirements, design and implementation move forward together.

Situation	Plan-driven	Agile
Very detailed specification and design before implementation	✓	
Deliver software to customers and get rapid feedback		✓
Large systems development	✓	
Systems require a lot of analysis before implementation	✓	
Long-lifetime system is required	✓	
Development may require up-to-date technologies for support		✓
Development team is distributed (outsourced)	✓	
Development team has strong rapid development skills		✓
The system is subject to regulation (external entity)	✓	

06 Scrum

Traditional project management suits Plan-driven development. Agile needs a different

approach, and the best known is **Scrum**. It runs in three phases:

#	Phase	What happens
1	**Initial phase**	We set the general goals and design the architecture.
2	**Sprint cycles**	Each cycle lasts 2 to 4 weeks and produces a new part of the system.
3	**Closure phase**	We close the project and complete the documents.

Sprint planning

It must happen before the sprint starts.

Inputs	Outputs
Team capacity Product backlog Business conditions Current product Technology	Sprint goal Sprint backlog

07 Sprint calculations

This is the calculation question of the lecture. There are three formulas that follow one

another.

1 SPRINT VELOCITY **Average completed points**

The average of the story points completed in the last 3 to 5 sprints.

```
Sprint Velocity = sum of completed story points ÷ number of sprints
```

2 FOCUS FACTOR **The team's ability to focus**

```
Focus Factor = Sprint Velocity ÷ (people × hours per day)
```

The ideal range is from `0.6` to `0.8`.

3 EFFECTIVE TEAM CAPACITY **Actual working hours**

```
Capacity = Focus Factor × (people × hours per day × working days)
```

A week has 5 working days. We leave out the weekend.

The slide example

A team of 5 people, 8 hours per day, and an average of 30 points. The next sprint is 3 weeks.

```
Focus Factor = 30 ÷ (5 × 8) = 30 ÷ 40 = 0.75
```

```
Working days = 3 × 5 = 15
```

```
Capacity = 0.75 × (5 × 8 × 15) = 0.75 × 600 = 450 hours
```

Good to know

Team capacity is the amount of work the team can put into the next sprint: **Team capacity = members count × productive days**. It is determined accurately after the third or fourth sprint, because by then we have real data about the team's performance.

08 Summary of the four models

	Waterfall	Incremental	Reuse-oriented	Agile
Requirements volatility	Low	Medium	Low	High
Project size	Large	Medium	Any	Small
Customer involvement	Low	Medium	Low	High
Technical risk	Low	High	Medium	High
Release schedule	Long	Medium	Short	Very short

Memorize it

This table is the key to the model selection question, which appeared in both midterms for 4 marks.

09 How does it appear in the midterm?

The calculation question (first midterm · 4 marks)

Team of 6 people; workday = 8h. Last 4 sprints completed story points: 22, 28, 26, 24.

```
1) Sprint Velocity = (22 + 28 + 26 + 24) ÷ 4 = 100 ÷ 4 = 25
```

```
2) Focus Factor    = 25 ÷ (6 × 8) = 25 ÷ 48 = 0.52
```

```
3) Working days    = 3 weeks × 5 = 15
```

```
   Capacity        = 0.52 × (6 × 8 × 15) = 0.52 × 720 ≈ 375 hours
```

The Focus Factor here is `0.52`, below the ideal range.

Part 4: the next backlog is 30 points, is the plan feasible? The solved copy chose **Prudent**, because 30 is greater than the team's average of 25. The rule written in it is:

Comparison	Verdict
The backlog is close to the velocity	**Feasible**
The backlog is slightly larger	**Prudent**
The backlog is much larger	**Not feasible**

Choosing the model (4 marks in each midterm)

If you see this in the project description	Choose
Customer available daily, requirements that change a lot, a small team in one place, a short time frame.	**Agile**
Fixed requirements, a large or distributed team, documents and sign-offs, regulations and laws.	**Plan-Driven**
Words such as COTS, vendor SDKs or integrate, which means ready-made components.	**Component-Based**

The projects of the first midterm and their answers:

Project	Answer	Reason
Smart-City Sensor Hub	**Component-Based**	It must integrate with COTS gateways and vendor SDKs.
Banking Regulatory Reporting	**Plan-Driven**	Requirements fixed by regulations, 30 people, documents and sign-offs.
Campus Social-Learning MVP	**Agile**	Daily feedback, 6 developers in one place, launch within weeks.
Medical Device Companion App	**Plan-Driven**	Medical standards and mandatory life-cycle documents.

10 Common mistakes

Mistake	Correct
Dividing the velocity by the number of people only	Divide by (people × hours per day).
Counting the week as 7 days	A week has 5 working days.
Forgetting to multiply by the Focus Factor	Capacity = FF × (people × hours × days).
Mixing up Refactoring and Pair programming	**Refactoring** is improving the code. **Pair programming** is two programmers working together.
Choosing Agile for a project with regulations and documents	This is **Plan-Driven**.

11 Test yourself

1. A team of 4, an 8-hour workday, last 5 sprints: 18, 22, 20, 24, 21. Calculate the Velocity and the Focus Factor.
2. For the same team, calculate the Capacity for a sprint that lasts two weeks.
3. Which XP practice means integrating every completed task after the tests pass?
4. A project: 5 developers in one office, the customer gives feedback daily, and the features change every week. Which model?
5. What are the three parts of a User story?

Answers

```
1) Velocity = (18 + 22 + 20 + 24 + 21) ÷ 5 = 105 ÷ 5 = 21

   Focus Factor = 21 ÷ (4 × 8) = 21 ÷ 32 = 0.656

2) Working days = 2 × 5 = 10

   Capacity = 0.656 × (4 × 8 × 10) = 0.656 × 320 = 210 hours
```

#	Answer
3	**Continuous integration**.
4	**Agile**: a small team, an available customer, and changing requirements.
5	**Who**, **What** and **Why**.